Museum Heist - Technical Defense Guide

Purpose: provide a precise explanation of how the implemented code works and how to defend it during the live demonstration.

System Flow

The game starts in game/ui/main.py. Pygame receives keyboard input. The UI writes an action to input.json using bridge.py. The bridge runs the C++ executable. The C++ engine validates rules and writes state.json. Python loads that state, recomputes greedy and backtracking results, and redraws the screen.

Movement Event

When the user presses WASD or an arrow key, Python writes action=move, direction, previous player position, movement history, collected items, score, alarm, elapsed time, and flags. C++ computes the next cell and checks grid boundaries, walls, cameras, guards, and vision zones. If valid, the move is appended to MovementList. If unsafe, game_over can be triggered.

C++ Engine

engine/main.cpp is the authoritative rules layer. It selects a map by difficulty, computes vision zones, validates movement, processes score and item collection, enforces alarm, movement, and time limits, and writes state.json.

MovementList

MovementList is a custom linked list. Each node stores row, col, step, and next. It records valid movement history. push_back is O(1) because the list stores a tail pointer. Traversal and clear are O(n).

ItemBST

ItemBST stores remaining items ordered by value. Smaller values go left and larger values go right. Inorder traversal writes items_sorted_by_value in ascending order. Insert is O(log n) average and O(n) worst case.

Greedy Algorithm

choose_greedy_item(state) highlights one item using local criteria: value, guard proximity, and vision-zone danger. It is a recommendation, not a global optimizer. Complexity is O(i(g+v)).

Backtracking Algorithm

find_escape_route(state) uses recursive choose-explore-unchoose logic. It avoids walls, cameras, guards, vision zones, and visited cells. Worst-case complexity is exponential, approximately O(4^n), but practical for the implemented grids.

JSON Bridge

input.json represents the action and preserved state. state.json represents the complete state written by C++. JSON was chosen because it is readable, easy to debug, and satisfies the file-based integration constraint.

Demo Sequence

1) Show main menu and difficulty selection. 2) Open tutorial with H. 3) Start game with Enter. 4) Move using WASD. 5) Show greedy highlight and backtracking route. 6) Show item collection. 7) Trigger a vision-zone game over. 8) Restart with R. 9) Return to menu with M. 10) Reach exit if time allows.

Core Questions

Why linked list? Dynamic movement history. Why BST? Ordered item output. Why greedy? Local item recommendation. Why backtracking? Safe route search under constraints. Why JSON? Simple cross-language bridge.

Test and Validation Summary

Area	What to demonstrate
Compilation	g++ builds engine/museum_engine without errors.
Pygame launch	python game/ui/main.py opens the main window.
Movement	Player moves one cell and movement history increases.
Invalid moves	Wall collision keeps position and increases alarm.
Items	Collected item increases score and disappears.
Vision	Entering vision zone causes Game Over.
Limits	Alarm, movement, and time limits can cause loss.
Escape	Reaching exit shows final metrics and rank.
Controls	R restarts, M returns to main menu, H opens tutorial.